

Nmap Enumeration Lab (v3) — Full Port Scan + OS Detection su Metasploitable2

Tool utilizzato:

Nmap Target: Metasploitable2 (192.168.1.16)

Ambiente: Kali Linux + Metasploitable2, VM UTM su Apple Silicon

Nota etica/metodologica: come nei lab precedenti, Metasploitable2 è una VM deliberatamente vulnerabile distribuita a scopo didattico. Il lab si è fermato alla fase di enumeration, nessun exploit eseguito.

Obiettivo:

Approfondire ulteriormente l'enumerazione già condotta su Metasploitable2 (vedi lab precedente), stavolta rimuovendo il limite di default di Nmap alle sole top 1000 porte, per verificare se servizi aggiuntivi restano nascosti su porte meno comuni — e aggiungendo il rilevamento del sistema operativo.

Perché una scansione precedente poteva non bastare:

Il comando `-sV` usato nei lab precedenti, senza specificare altro, scansiona di default solo le 1000 porte più comuni su un totale di 65.535 disponibili (TCP). Questo è sufficiente per una prima ricognizione rapida, ma può lasciare fuori servizi configurati su porte non standard — motivo per cui in questo lab si è deciso di ripetere la scansione con il flag `-p-` (tutte le porte) e `-O` (OS fingerprinting).

Comando eseguito:

```
sudo nmap -sV -sC -O -p- 192.168.1.16
```

Spiegazione dei flag:

- `-sV` → version detection (già noto dai lab precedenti)
- `-sC` → script NSE di default (già noto)
- `-O` → tenta di identificare il sistema operativo del target tramite fingerprinting dello stack TCP/IP

- **-p-** → scansiona tutte le 65.535 porte TCP, non solo le prime 1000

Tempo di esecuzione: 140 secondi (contro i ~64 secondi della scansione precedente limitata alle top 1000) – conferma diretta del costo in tempo di una scansione completa.

Risultato: 30 porte aperte (vs 23 rilevate in precedenza)

Rispetto al lab precedente, la scansione completa ha rivelato 7 porte aggiuntive, tutte su numerazioni alte non incluse nel default di Nmap:

Porta 3632 – distccd v1

Storicamente associato a una vulnerabilità di esecuzione codice remoto ben nota (distcc, se mal configurato, può eseguire comandi arbitrari da remoto)

Porta 6697 – UnrealIRCd (seconda istanza)

Stessa applicazione già vista su 6667, in ascolto anche su porta alternativa

Porta 8787 – Ruby DRb RMI

Un altro servizio storicamente a rischio: DRb (Distributed Ruby) senza restrizioni può consentire esecuzione di codice Ruby arbitrario da remoto

Porta 36518 – mountd (RPC)

Servizio ausiliario NFS, gestisce le richieste di mount

Porta 43944 – nlockmgr (RPC)

Servizio ausiliario NFS per la gestione dei lock sui file

Porta 48658 – status (RPC)

Servizio ausiliario RPC (rpc.statd)

Porta 49407 – java-rmi (duplicato)

Stessa applicazione già vista su 1099, su porta alternativa registrata dinamicamente

Osservazione chiave: due di questi (distccd e Ruby DRb) sono servizi con precedenti storici di vulnerabilità critiche di esecuzione codice da remoto – nessuno dei due sarebbe stato visibile con una scansione di default limitata alle top 1000 porte. Questo dimostra concretamente perché, in una valutazione di sicurezza reale, affidarsi solo allo scan rapido può dare un falso senso di sicurezza ("ho controllato tutto") quando in realtà porte critiche restano fuori dal radar.

OS Detection (-O)

Device type: general purpose Running: Linux 2.6.X OS details: Linux 2.6.9 - 2.6.33

Il fingerprinting ha identificato correttamente un kernel Linux della serie 2.6 (compatibile con l'età nota di Metasploitable2, rilasciata su base Ubuntu 8.04, circa 2008) – utile per contestualizzare quanto sia datato l'intero ambiente e, di conseguenza, quanto siano prevedibili le vulnerabilità presenti.

Findings già noti confermati (dai lab precedenti)

Tutti i findings precedenti restano confermati e coerenti: FTP anonimo su vsftpd 2.3.4, bindshell su 1524, SMB message signing disabilitato, SSLv2/certificati scaduti su SMTP e PostgreSQL, salt MySQL esposto. Per il dettaglio completo si rimanda al writeup precedente.

Conclusioni:

Questo giro di scansione ha avuto un valore didattico specifico e distinto dal lab precedente: non tanto "trovare più vulnerabilità" quanto capire i limiti metodologici di una scansione di default. La differenza tra 23 e 30 porte aperte, in questo caso, ha significato la differenza tra vedere o non vedere due servizi storicamente critici (distccd, Ruby DRb).

Cosa ho imparato:

- Il flag **-p-** non è "opzionale per completezza", ma può essere determinante nel trovare servizi ad alto rischio su porte non standard
- Il costo in tempo (140s vs 64s su un solo host) di una scansione completa – su reti più grandi questo si traduce in ore, da pianificare di conseguenza
- Uso di **-O** per il fingerprinting del sistema operativo, utile per contestualizzare l'età e la prevedibilità delle vulnerabilità di un target
- Riconoscere, dal solo nome del servizio, quali porte "alte"/non standard meritano attenzione prioritaria (distccd, DRb) rispetto a quelle che sono solo porte ausiliarie di servizi già noti (mountd, nlockmgr, status – tutti satelliti di NFS/RPC)

Lab correlato: Nmap Enumeration Lab v2 – Metasploitable2, scansione standard